

Apex Rover: Dual-Mode Wi-Fi & Autonomous Rover

Design for Manufacturing (DFM) & Technical Engineering Report

Author: Mickael Bah

Date: August 2026

Revision: 2.1 — Merged Portfolio Edition (DFM Process, Manufacturing Lessons & Reproducibility)

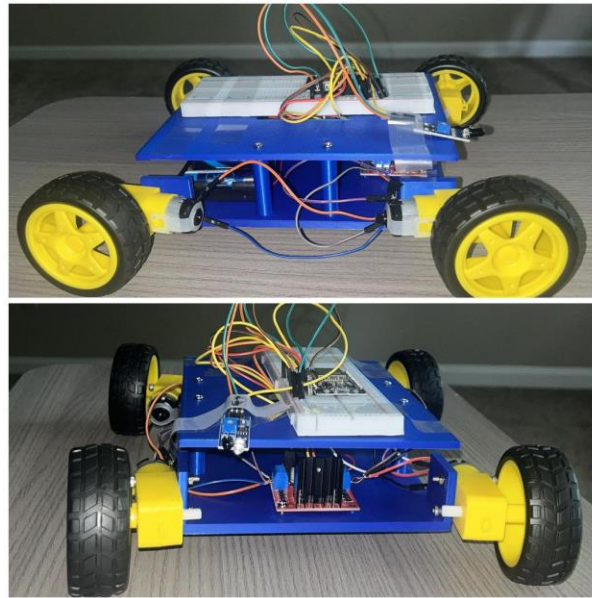


Figure 1.1: Completed Apex Rover hardware build featuring dual-deck 3D printed chassis and ESP32 control system.

Project purpose. This project was undertaken not only to produce a functional rover, but to gain hands-on experience with a complete engineering workflow: defining requirements, developing a manufacturable mechanical design, fabricating the parts, assembling and testing the physical prototype, identifying manufacturing/design weaknesses, and using those observations to develop a V2 design.

1. Executive Summary

The Apex Rover is a custom 4WD robotic vehicle built around a dual-deck 3D-printed chassis. It provides wireless manual control through an ESP32-hosted web interface, and an autonomous obstacle-avoidance mode using an active infrared sensor combined with a periodic patrol-turn pattern so the rover explores rather than driving in a single straight line. The mechanical system was intentionally developed as a practical Design for Manufacturing (DFM) exercise using FDM 3D printing and readily available hardware.

The engineering value of the project lies in the transition from CAD to a physical prototype. Manufacturing introduced real-world considerations that are difficult to fully predict in CAD, including printed-hole fit, joint rigidity, accessibility of fasteners, cable routing, and power-system behavior under combined electrical load. These observations were used to define specific V2 improvements.

The final design should be read as a validated V1 prototype rather than a claim of production-ready perfection. This report documents the design rationale, fabrication strategy, electrical architecture, assembly process, verification observations, and proposed V2 revisions — with enough pinout, wiring, and firmware detail that another builder could reproduce the prototype.

2. Design Requirements & Engineering Objectives

The following requirements define the engineering target for the prototype and provide a basis for evaluating whether individual design decisions serve the overall purpose of the rover.

Requirement	Target / Objective
Drive configuration	4WD skid-steer
Operating modes	Wi-Fi manual control and autonomous obstacle avoidance
Mechanical manufacturing	FDM 3D printing for custom chassis components
Custom parts	Minimize the number of unique fabricated components
Chassis architecture	Two-level deck separating drive hardware from control electronics
Fasteners	Standard M3 hardware where practical
Chassis footprint	Approximately 180 mm x 140 mm
Deck separation	40 mm integrated structural pillars
Design objective	Low-cost, reproducible prototype with accessible assembly
Learning objective	Experience the complete design -> manufacture -> test -> redesign cycle

3. Design Development & Concept Selection

The chassis architecture was developed around a practical manufacturing constraint: the rover needed to contain motors, battery/power hardware, the ESP32, breadboard, and sensors while remaining printable and easy to assemble. The final architecture uses two structural levels. The lower deck carries the higher-current drive hardware, while the upper deck carries the control electronics and sensor system.

Design logic: separating the two functional zones improves organization, protects the control electronics from the motor area, and creates a dedicated path for wiring between decks. The final chassis also keeps the number of custom printed components low — the primary fabricated structure consists of just two parts, P-01 and P-02.

This project should be understood as an iterative engineering process rather than a single CAD-to-print operation. The physical V1 prototype was used to expose limitations and inform the V2 design, detailed in Sections 9 and 10.

4. Bill of Materials & Manufactured Components

Category	Component	Specification / Qty	Purpose	Est. Unit Cost*
OTS	ESP32 NodeMCU	30-pin, 3.3V logic, Soft-AP Wi-Fi, 1	Control + web host	\$6-10
OTS	L298N motor driver	Dual H-bridge, 2A peak, 1	Motor power/control	\$3-6
OTS	TT gearbox motors	3V-6V DC, 1:48 ratio, 4	4WD propulsion	\$1-2 ea.
OTS	IR obstacle module	Active IR transceiver, 1 (front-facing)	Obstacle detection	\$1-3
OTS	Battery enclosure	6x AA, 9V nominal series input, 1	System power	\$2-4

Category	Component	Specification / Qty	Purpose	Est. Unit Cost*
OTS	Solderless breadboard	830-point dual rail grid, 1	Signal distribution & bus grid	\$3-5
OTS	M3 fasteners	M3x10 / M3x30, 12	Structural mounting	\$3-5 (set)
Printed	P-01 Top deck	180x140x3.5 mm; 20% gyroid	Upper structure	filament only
Printed	P-02 Lower chassis	180x140 mm; 40 mm pillars; 20% gyroid	Lower structure + pillars	filament only

5. DFM Strategy for FDM Manufacturing

The chassis was designed specifically for FDM manufacturing rather than treating 3D printing as a simple export step. The most important considerations were dimensional tolerance, fastener fit, joint design, printability, assembly access, and cable management.

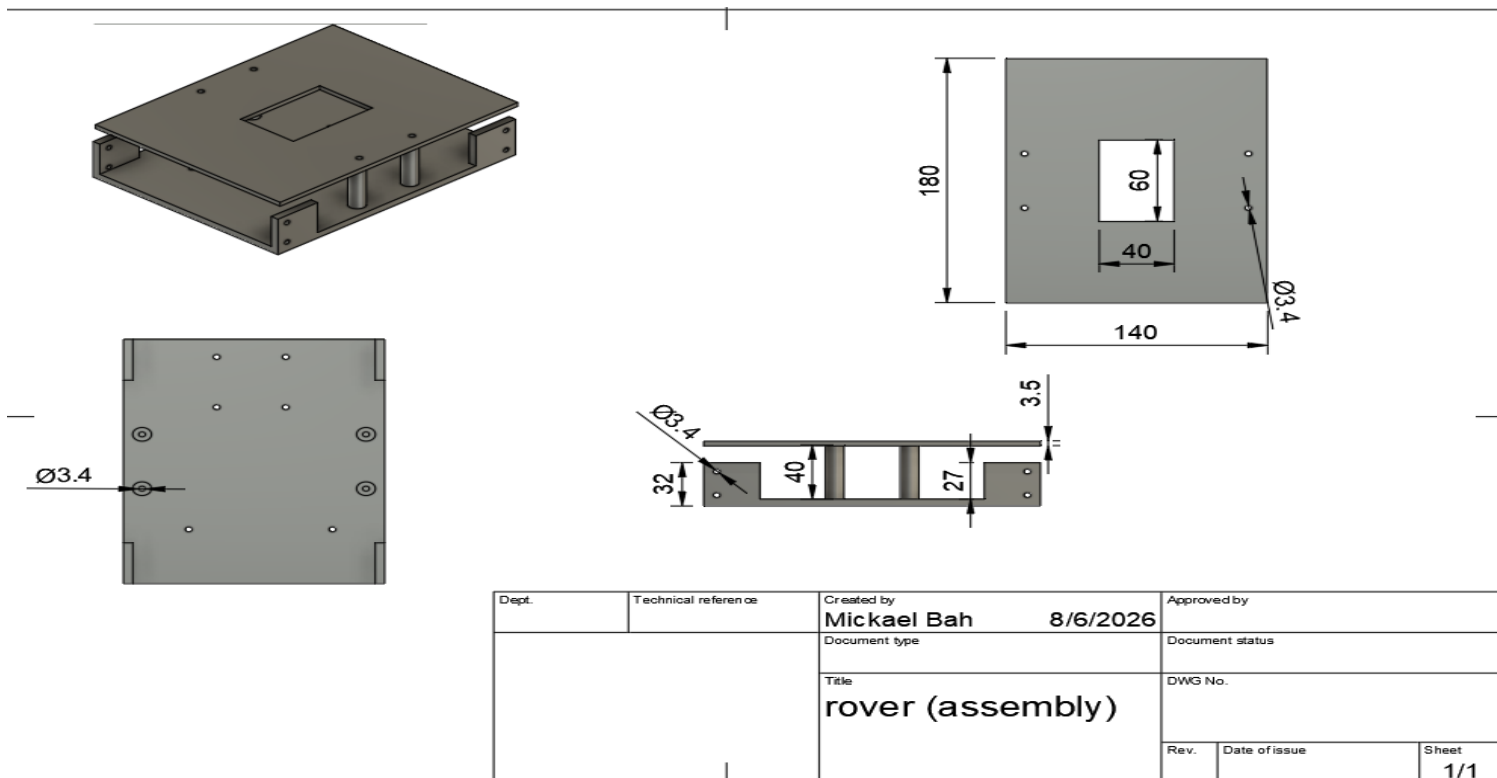


Figure 5.1: 2D dimensioned drawing sheet detailing deck plate dimensions (180 mm x 140 mm), 40 mm pillar heights, center cutout (60 mm x 40 mm), and Ø3.4 mm hole callouts.

5.1 Fastener Hole Tolerancing

M3 clearance holes were modeled at Ø3.4 mm rather than the nominal 3.0 mm bolt diameter. This provides clearance for the printed part and accounts for the dimensional behavior of FDM holes: inner diameters typically contract 0.1-0.2 mm during cooling. The resulting design allows the M3 bolt to pass through without binding. The V1 build confirmed that the clearance strategy works well for pass-through fastening, but also demonstrated that a clearance hole cannot itself provide a threaded connection without a nut or insert.

5.2 Structural Joint Design

The lower chassis incorporates 40 mm pillars to support the upper deck. This reduces the number of separate structural parts and simplifies assembly. However, the physical prototype showed minor rotational wobble at the pillar joints during high-torque skid turns. This is a useful DFM lesson: a geometry that is adequate for assembly in CAD can still require additional mechanical retention when subjected to dynamic loads.

5.3 Cable Management & Assembly Access

A 60x40 mm central cable aperture was incorporated into the upper deck to route motor and power wiring between the two decks. This feature reduces cable interference and keeps wiring organized without requiring additional printed cable-management components.

5.4 Material, Infill & Print Strategy

The documented V1 print strategy uses PLA or PETG at 20% gyroid infill. The final report records these settings as the validated baseline. Print-time, filament-mass, and cost measurements were not captured during the V1 build; these should be added from slicer history once available (see Appendix A).

6. Electrical Architecture & Wiring Matrix

The rover integrates mechanical, electrical, and software subsystems on a common ground bus and a single power rail distributing power from the battery enclosure to the motor driver, ESP32, and sensor.

6.1 Power Budget & Voltage Rationale

Design decision - why 9V instead of 6V:

The battery pack supplies all three subsystems in parallel off one rail: the L298N/motors, the ESP32, and the IR sensor. A 4xAA (6V) pack is sufficient for the TT motors alone, but once the ESP32's Wi-Fi radio (which draws current in bursts well above its average load during transmission) and the IR sensor share the same rail, the combined draw pulls the pack voltage down enough that the ESP32 browns out or resets mid-drive. The 6-cell (9V nominal) pack was chosen to keep enough headroom under combined load so the ESP32 stays powered through Wi-Fi transmit bursts. The tradeoff is that the motors (rated 3-6V) are run above their nominal rating.

- Motors run hotter and with reduced service life versus their 3-6V rating; acceptable for a prototype/demo.
- The L298N has its own onboard 5V regulator (enabled via the board's jumper) that derives logic-side 5V from the same input rail.
- V2 candidate fix: a dedicated 5V regulator feeding the ESP32 and IR sensor off a separate, current-limited tap — isolates the microcontroller from motor-induced voltage sag entirely rather than relying on pack headroom.

6.2 Wiring / Block Diagram

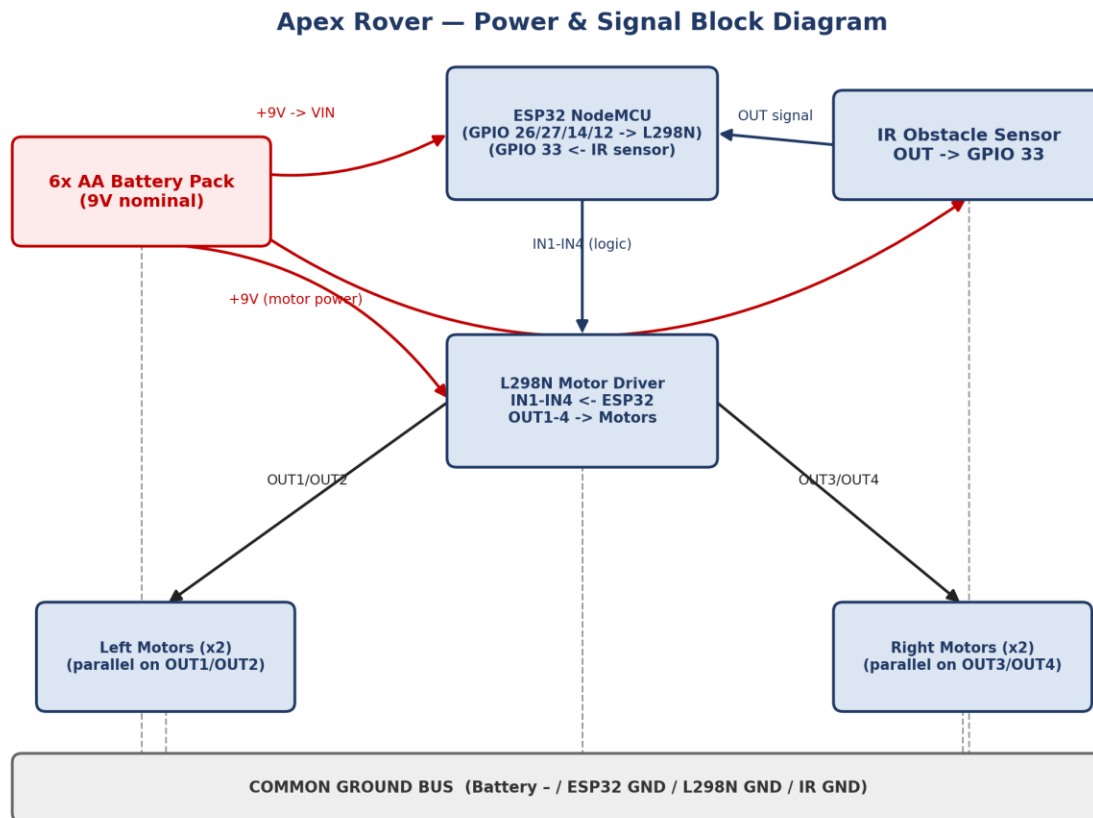


Figure 6.1: Power and signal block diagram. Red lines are the shared 9V power rail; navy lines are logic/signal connections; dashed grey lines show each subsystem's ground return to the common ground bus.

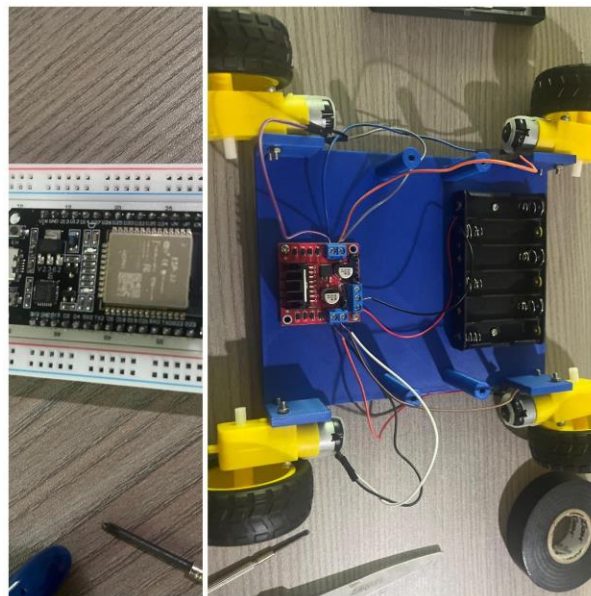


Figure 6.2: Physical wiring distribution on the lower deck, showing the ESP32/breadboard connections and the L298N/battery power rails.

6.3 Master Pinout Connections Table

Source Device	Output Pin	Target Component	Input Pin	Wire Signal / Function
L298N Driver	OUT1 / OUT2	Left Motors (Parallel)	Motor Terminals	Left Drive Output
L298N Driver	OUT3 / OUT4	Right Motors (Parallel)	Motor Terminals	Right Drive Output
ESP32	GPIO 26	L298N Driver	IN1	Left Motors Forward
ESP32	GPIO 27	L298N Driver	IN2	Left Motors Reverse
ESP32	GPIO 14	L298N Driver	IN3	Right Motors Forward
ESP32	GPIO 12	L298N Driver	IN4	Right Motors Reverse
ESP32	GPIO 33	IR Sensor Module	OUT	Obstacle Signal Input
Breadboard Rail	GND (-)	ESP32 / L298N / IR	GND	Common Ground Bus
Breadboard Rail	RED (+)	ESP32 VIN / IR VCC	-	Common 9V Power Bus

7. Manufacturing & Assembly Process

The manufacturing workflow was intentionally organized as a reproducible sequence: fabricate the structural parts first, install the mechanical system, install the electronics, route wiring, then perform electrical and functional checks before final operation.

Stage	Process / DFM Consideration
1	Print P-01 and P-02 using the documented 20% gyroid strategy.
2	Install the four TT motors on the lower chassis; position left/right motor pairs symmetrically for skid-steer.
3	Install the upper deck using M3 x 30 mm fasteners through the Ø3.4 mm clearance holes.
4	Place the L298N near the motors to keep high-current wiring runs short.
5	Mount ESP32, breadboard, and IR sensing hardware on the upper deck.
6	Connect left and right motor pairs to the L298N outputs (OUT1/OUT2, OUT3/OUT4).
7	Connect ESP32 control GPIOs to the L298N per the master pinout (Section 6.3).
8	Connect the IR sensor and shared power/ground rails.
9	Route power and signal wiring through the central 60x40 mm deck aperture.
10	Power up with wheels off the ground and verify each direction command drives the correct motor the correct way before final operation.

8. Electrical & Software Integration

The ESP32 provides the control interface and hosts a Wi-Fi access point. Manual commands are sent through an HTML5 interface via JavaScript fetch() calls attached to ontouchstart/ontouchend events for Hold-to-Drive control, while autonomous operation uses a non-blocking state machine.

The documented firmware architecture includes DRIVING, AVOID_STOPPING, AVOID_REVERSING, AVOID_PAUSING, AVOID_TURNING, and PATROL_TURNING states. The use of millis()-based state transitions avoids blocking delay() calls, allowing the web interface to remain responsive during autonomous maneuvers.

8.1 Development Environment Setup

1. Install the Arduino IDE (2.x recommended).
2. Add the ESP32 board package: File > Preferences > Additional Boards Manager URLs, add: https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
3. Tools > Board > Boards Manager: search "esp32" (Espressif Systems) and install.
4. Tools > Board: select the profile matching your NodeMCU-32S/ESP32 Dev Module variant.
5. No external libraries are required beyond the built-in WiFi.h that ships with the ESP32 board package.
6. Select the correct COM/serial port, set baud rate to 115200 to match Serial.begin(), and upload.

8.2 Web Interface



Figure 8.1: Web interface running in landscape mode, showing the AUTO/MANUAL mode toggle and Hold-to-Drive gamepad controls.

8.3 Firmware Source

The embedded control software is compiled using the Arduino ESP32 core. HTTP REST endpoints (/forward, /backward, /left, /right, /stop, /auto, /manual) drive both manual gamepad driving and state switching. A DEBUG_SENSOR flag prints raw IR transceiver readings to Serial to verify signal polarity and detection threshold during bring-up. The complete source code (apex_rover.ino) is maintained in the project repository referenced in Appendix B.

9. Physical Prototype Verification - V1

The most important verification result was not simply that the rover operated; it was that physical testing revealed design behavior that could be improved. These observations form the basis for the V2 engineering roadmap in Section 10.

Observed Result	Engineering Interpretation	Design Implication
Minor rotational wobble at pillar joints during high-torque skid turns	Friction-fit structural connection is not sufficiently rigid for dynamic loading	Use positive mechanical retention such as heat-set inserts
Ø3.4 mm holes provide easy M3 pass-through	Clearance-hole strategy is appropriate for bolt-through joints	Retain Ø3.4 mm for clearance applications; add nut/insert features where threading is needed
M3 machine screws cannot tap directly into Ø3.4 mm clearance holes	Clearance geometry and threaded geometry serve different functions	Use a smaller self-tapping profile or a separate insert/nut strategy
Motor and logic systems share one power architecture	Motor loads can create electrical disturbances or voltage sag	V2 should isolate logic power with a dedicated regulator

10. V1 → V2 Engineering Revision

The V2 roadmap is derived directly from the physical prototype. Each change is linked to an observed problem rather than being added only for complexity or appearance.

V1 Condition	Likely Cause / Lesson	V2 Revision
Pillar joint wobble	Friction-fit joint lacks positive retention under torque	M3 heat-set brass inserts in pillar tops
Clearance hole used where threading is desired	Ø3.4 mm is appropriate for clearance, not direct M3 thread formation	Ø2.8 mm self-tapping holes where appropriate
Nut access can complicate assembly	External nuts require tool access and can interfere with surfaces	5.5 mm recessed hex-nut pockets
Logic and motor power share the system supply	Motor current can introduce voltage disturbances	Dedicated 5V regulator for ESP32 + IR sensor
Prototype documentation separate from controlled manufacturing files	Reproducibility requires a clear file/configuration baseline	Maintain revision-controlled CAD, drawings, BOM, and firmware manifest

11. Lessons Learned from the DFM Process

The primary objective of this project was hands-on exposure to the design-for-manufacturing cycle. Several practical lessons emerged from taking the design from CAD into a physical prototype:

- Nominal dimensions are not always manufacturing dimensions. FDM holes and interfaces require intentional tolerance design.
- CAD cannot replace physical validation. The pillar wobble was identified during operation rather than from the CAD model alone.
- Fastening strategy must be designed around the manufacturing process. Clearance holes, self-tapping holes, nuts, and threaded inserts solve different problems.
- Assembly is part of DFM. Cable access, fastener access, and component placement affect whether a design is practical to build and service.
- Prototype failures are design data. The V1 weaknesses became direct inputs to the V2 design rather than being treated simply as defects.
- Good DFM is iterative. The goal of the prototype was not to produce a perfect first iteration, but to learn how manufacturing behavior should influence the next design.

12. Conclusion

The Apex Rover successfully served as a hands-on Design for Manufacturing and robotics engineering project. The completed prototype demonstrates the integration of mechanical design, FDM fabrication, electrical wiring, embedded control, wireless communication, and autonomous behavior.

More importantly, the project provided direct experience with the transition from digital design to physical manufacturing. The V1 prototype exposed real limitations in joint rigidity, fastening strategy, and power architecture. These observations were converted into specific V2 design changes, including heat-set inserts, differentiated hole profiles, recessed nut pockets, and dedicated logic regulation.

The resulting design should be considered a validated V1 prototype with a documented path toward a more robust V2. The project demonstrates the engineering mindset required for DFM: design with manufacturing constraints in mind, build the part, test it under real conditions, learn from the result, and feed those lessons back into the next revision.

Appendix A — Supplementary Engineering Deliverables

The following supplementary engineering assets accompany this documentation package:

- 2D Dimensioned Engineering Drawings (Figure 5.1 - Included)
- Chassis Part STL & STEP Files (P-01 & P-02)
- As-Built Wiring & Circuit Schematics (Figure 6.1 & 6.2 - Included)
- Smartphone Gamepad Interface Layout (Figure 8.1 - Included)

Appendix B — Firmware Source

The embedded control software for the ESP32 is compiled using the standard Arduino IDE framework. The source code utilizes non-blocking state routines, HTTP REST endpoints, and an integrated HTML5/JavaScript gamepad engine.

- **Primary Code File:** `apex_rover.ino`
- **Target Microcontroller:** ESP32 NodeMCU / Dev Module
- **Development Environment:** Arduino IDE 2.x (Espressif ESP32 Core)
- **Dependencies:** `WiFi.h` (Standard ESP32 Built-in Library)